

Hierarchical Structure for Neurofitness Training Programs

Based on the documents, I can identify the main program categories, workout types, and specific training sessions within the Neuronline app. Let me organize them hierarchically:

1. Program Categories

- **Нейрозарядка** (Neural Warm-up)
 - *Purpose:* "Зарядиться энергией и проснуться", "Снизить уровень стресса", "Успокоиться и восстановить силы"
 - *Видео:* "Бодрость #1", "Бодрость #2", "Бодрость #3", "Бодрость. Мозжечок", etc.
- **Нейротренировки** (Neuro Workouts)
 - *Purpose:* "Улучшить работу мозга", "Продуктивность работы мозга"
 - *Видео:* "Нейротренировка #11", "Нейротренировка #12", etc.
- **Добавь энергии** (Energy Boost)
 - *Purpose:* "Повысить двигательную активность", "Стать более выносливым", "Стать более энергичным"
 - *Видео:* "Full body с резинкой", "Интенсивная", "Плиометрика", etc.
- **Идеальные ягодичы** (Perfect Glutes)
 - *Purpose:* "Идеальные ягодичы и красивые ноги"
 - *Видео:* "Нейроягодицы", "Ягодицы 3D", "Координационная тренировка на ягодичы", etc.
- **Пресс и дыхание** (Abs and Breathing)
 - *Purpose:* "Плоский живот и узкая талия", "Подтянутая фигура"
 - *Видео:* "Круговая на пресс", "Дыхание для восстановления", etc.
- **Красивая осанка** (Beautiful Posture)
 - *Purpose:* "Красивая осанка и подвижные суставы", "Развить гибкость тела", "Избавиться от боли в суставах"
 - *Видео:* "Мобильность на все тело", "Мягкая мобильность позвоночника", etc.

2. Professional Development Programs

- **Executive Cognitive Enhancement Program**
 - *Components:* Morning Activation, Midday Optimization, Evening Recovery
- **Visual-Motor Integration Program**
 - *Components:* Visual Training, Motor Training, Integration Protocols
- **Stress Resilience and Recovery Program**
 - *Components:* Physiological Regulation, Stress Response Management, Cognitive Resilience
- **Athletic Performance Enhancement Program**
 - *Components:* Power Development, Neural Drive

3. Training Implementation Framework

- **Development Phases**
 - Neural Adaptation (2 weeks)
 - Integration (3 weeks)
 - Performance (3 weeks)

- **Recovery Integration Protocol**

- Daily Maintenance
- Intensive Recovery

Let me use the MCP tools to create a decision tree for recommending the appropriate neurofitness program based on user goals and needs.

View result from macos_systeminfo from toolbase-proxy (local)

```
{ `category`: `all` }
```

```
No result received from client-side tool execution.
```

I'll create a decision tree to help determine the most appropriate neurofitness program based on user goals, current state, and needs. This will form a logical structure for program recommendation.

Decision Tree for Neurofitness Program Recommendations

1. What is your primary goal?

- **Cognitive and Brain Function Improvement**
 - *Are you looking for overall brain function improvement?*
 - Yes → Нейротренировки (Neuro Workouts)
 - No → Continue
 - *Are you an executive looking for better cognitive performance?*
 - Yes → Executive Cognitive Enhancement Program
 - No → Continue
 - *Do you need improved visual-motor coordination?*
 - Yes → Visual-Motor Integration Program
 - No → Continue
- **Energy and Physical Activity**
 - *Do you want to feel more energetic in the morning?*
 - Yes → Нейрозарядка (Neural Warm-up) - "Зарядиться энергией и проснуться"
 - No → Continue
 - *Do you want to become more enduring and energetic overall?*
 - Yes → Добавь энергии (Energy Boost)
 - No → Continue
 - *Are you an athlete looking to enhance performance?*
 - Yes → Athletic Performance Enhancement Program
 - No → Continue
- **Stress Reduction and Recovery**
 - *Do you need to reduce stress levels?*
 - Yes → Нейрозарядка - "Снизить уровень стресса"
 - No → Continue
 - *Do you need comprehensive stress management and recovery?*
 - Yes → Stress Resilience and Recovery Program
 - No → Continue
 - *Do you need to calm down and recover strength?*

- Yes → Нейрозарядка - "Успокоиться и восстановить силы"
 - No → Continue
 - **Body Shape and Aesthetics**
 - *Are you focusing on glutes and legs?*
 - Yes → Идеальные ягодицы (Perfect Glutes)
 - No → Continue
 - *Do you want a flat stomach and narrow waist?*
 - Yes → Пресс и дыхание (Abs and Breathing)
 - No → Continue
 - *Do you want an overall toned figure?*
 - Yes → Combine "Пресс и дыхание" and "Нейротренировки"
 - No → Continue
 - **Posture and Joint Health**
 - *Do you want better posture and mobile joints?*
 - Yes → Красивая осанка (Beautiful Posture)
 - No → Continue
 - *Do you need to develop body flexibility?*
 - Yes → Красивая осанка - "Развить гибкость тела"
 - No → Continue
 - *Do you need to eliminate joint pain?*
 - Yes → Красивая осанка - "Избавиться от боли в суставах"
 - No → Continue
2. **What is your current level of experience?**
- Beginner → Start with Basic Level programs
 - Intermediate → Consider Integration Phase programs
 - Advanced → Implement Performance Phase programs
3. **How much time can you dedicate daily?**
- 15-30 minutes → Focus on targeted, shorter sessions
 - 30-60 minutes → Standard program length
 - 60+ minutes → Consider combining multiple program elements
4. **Do you have any health restrictions?**
- Yes → Adapt programs based on specific restrictions
 - No → Proceed with standard recommendations
5. **Program Implementation**
- **Week 1-2: Neural Adaptation Phase**
 - Focus on pattern establishment
 - Weekly assessment of movement quality and neural efficiency
 - **Week 3-5: Integration Phase**
 - Focus on pattern refinement and increased loading
 - Bi-weekly assessment of movement complexity and cognitive load
 - **Week 6-8: Performance Phase**
 - Focus on skill expression under various conditions
 - Regular assessment of adaptability and recovery capacity

This hierarchical structure and decision tree should help organize the neurofitness program elements and guide users to the most appropriate training recommendations based on their specific needs and goals. The structure respects the existing video content names and categorizations from the provided documents.

You are an expert AI prompt engineer and writer with universal knowledge across various fields, known as the AI whisperer. Your task is to generate a comprehensive and structured JSON prompt for a Cursor.app AI agent, incorporating all necessary contexts and requirements for a Next.js 13+ enterprise fitness application. The application is built using Next.js 13+, TypeScript, Tailwind CSS, and Supabase for database management. It includes features like authentication, workout management, neurofitness programs, and gamification elements. - Create a JSON prompt that outlines the system architecture, database schema, component organization, routing structure, data flow, business logic, development standards, security policies, and maintenance procedures. - Ensure the prompt is detailed and comprehensive, covering all aspects of the application. - Use the provided file map and contents as a reference to ensure accuracy and completeness. - The JSON prompt must be formatted correctly and include all specified sections. - Ensure clarity and precision in the descriptions and specifications. - Do not include any placeholders or incomplete sections. Generate a JSON prompt for a Next.js 13+ application with TypeScript and Tailwind CSS. { "name": "Neurofitness Enterprise Application", "version": "3.0.0", "description": "Next.js 13+ enterprise fitness application with TypeScript, Tailwind CSS, Supabase integration, and comprehensive neurofitness features", "systemArchitecture": { "frontend": { "framework": { "name": "Next.js 13+", "routing": "App Router", "styling": "Tailwind CSS", "stateManagement": { "local": "React Hooks", "global": "Context API", "server": "Server Actions" } }, "components": { "organization": "feature-based", "structure": "atomic-design", "styling": "tailwind-modules" } }, "backend": { "database": { "provider": "Supabase PostgreSQL", "schemas": ["user_data", "content", "training", "metrics", "rewards"], "features": ["Full-text search", "Real-time subscriptions", "Row-level security", "Automated backups"] }, "authentication": "Supabase Auth", "storage": "Supabase Storage", "realtime": "Supabase Realtime" }, "databaseSchema": { "schemas": { "user_data": { "tables": ["users", "biometrics", "fitness_preferences", "available_days", "health_restrictions", "user_goals"], "views": ["active_users_view"] }, "content": { "tables": ["categories", "subcategories", "neuro_effects", "equipment", "courses", "videos"], "views": ["video_popularity_view"] }, "training": { "tables": ["user_courses", "fitness_plans", "weekly_plans", "daily_workouts", "workout_activities"], "views": ["user_workout_schedule_view"] } }, "appStructure": { "root": { "app": { "groups": { "(auth)": { "routes": ["/login", "/register", "/callback", "/signout"], "layout": "AuthLayout" }, "(dashboard)": { "routes": ["/dashboard", "/analytics", "/progress", "layout": "DashboardLayout" }, "(workouts)": { "routes": ["/workouts", "/workout-player", "/history", "layout": "WorkoutLayout"] } }, "components": { "features": { "achievements": ["AchievementGrid", "RewardsDisplay"], "workouts": ["WorkoutCard", "WorkoutProgress"], "neuro": ["NeuralTest", "CourseCard"] }, "layout": { "navigation": ["MainNav", "DashboardNav"], "containers": ["DashboardShell", "ContentWrapper"] } } }, "dataFlow": { "api": { "routes": { "base": "/api", "handlers": { "auth": ["login", "register", "callback"], "workouts": ["list", "detail", "progress"], "neuro": ["test", "courses", "results"] } } }, "state": { "contexts": { "auth": "AuthContext", "workout": "WorkoutContext", "rewards": "RewardsContext" } } }, "businessLogic": { "neurofitness": { "programTypes": { "neurocognitive": { "focus": "Neural pathway activation", "components": ["Vestibular", "Visual", "Coordination"], "progression": "4-week cycles" }, "postural": { "focus": "Alignment and stability", "components": ["Mobility", "Strength", "Balance"], "progression": "3-week cycles" } }, "adaptationMetrics": { "neural": ["Reaction time", "Movement precision", "Cognitive load tolerance"], "physical": ["Strength output", "Movement quality", "Recovery rate"] } }, "development": { "standards": { "typescript": { "strict": true, "paths": { "@/*": ["/src/*"] } }, "components": { "naming": "PascalCase", "structure": "atomic-design" }, "testing": { "unit": "vitest", "component": "testing-library", "e2e": "playwright" } }, "security": { "database": { "roles": ["neurofitness_admin", "neurofitness_app", "neurofitness_analyst"], "policies": { "rowLevelSecurity": true, "dataEncryption": true } } }, "maintenance": { "database": { "backups": { "schedule": "daily", "retention": "7 days" }, "optimization": { "vacuum": "weekly", "analyze": "daily" } } } } - Zero-Shot Chain of Thought Prompting: Break down the JSON structure into manageable sections to ensure comprehensive coverage. - Constrained Writing Techniques: Follow specific rules for JSON formatting to ensure consistency.

title: Neuronline Fitness System: Comprehensive Documentation
tags: [neuronline, fitness, programming, architecture, features]

System Overview

The Neuronline Fitness platform is a robust system designed to generate personalized fitness programs that integrate both neurological and physical training components. This documentation outlines the system's architecture, features, and implementation guidelines.

Core Architecture

Project Structure

neurofitness-app/	
app/	Next.js App Router
(auth)/	Authentication Group
/dashboard/	Dashboard Group
/workouts/	Workouts Group
(neuro)/	Neuro Features Group
(fitness)/	Fitness Group
(gamification)/	Gamification Group
(user)/	User Group
api/	API Routes
layout.tsx	Root Layout
components/	Reusable Components
features/	Feature-specific components
layout/	Layout components
ui/	UI components
common/	Common components
hooks/	Custom React hooks
lib/	Utility libraries
public/	Static assets
styles/	Global styles

Component Organization

components/	
features/	Feature-specific components
achievements/	Achievement-related components
workouts/	Workout-related components
neuro/	Neuro-related components
activity/	Activity tracking components
fitness/	Fitness program components
shop/	Shop components
layout/	Layout components

navigation/	Navigation components
[...layout components]	Other layout components
ui/	UI components
data-display/	Data display components
inputs/	Input components
feedback/	Feedback components
overlay/	Overlay components
navigation/	Navigation components
common/	Common components

Key Features

Personalized Program Generation

The system generates tailored fitness plans based on:

- User profiles and metrics
- Fitness goals and preferences
- Neurological testing results
- Physical capabilities and limitations

```
{
  "programGeneration": {
    "inputs": [
      "userProfile",
      "fitnessGoals",
      "neuroTestResults",
      "physicalMetrics",
      "trainingPreferences"
    ],
    "outputFormats": [
      "weeklySchedule",
      "dailyWorkouts",
      "exerciseInstructions",
      "progressionMetrics"
    ],
    "adaptationFactors": [
      "completionRates",
      "performanceMetrics",
      "userFeedback",
      "recoveryPatterns"
    ]
  }
}
```

Neural-Physical Integration

The system effectively balances neurological and physical training through:

```

{
  "neuralPhysicalIntegration": {
    "components": [
      {
        "name": "Vestibular System Training",
        "focus": "Balance and spatial awareness",
        "exercises": ["Balance drills", "Coordination patterns", "Spatial orientation
exercises"]
      },
      {
        "name": "Visual System Training",
        "focus": "Visual processing and eye movement",
        "exercises": ["Eye tracking exercises", "Visual memory tasks", "Depth perception
drills"]
      }
    ]
  }
}

```

title: Neuronline Fitness Program Decision Tree

tags: [neuronline, fitness, decision tree, programming, cognitive enhancement, physical preparation]

Neuronline Fitness Program Selection Decision Tree

Start

1. Assess Neural Readiness

- **Low**
 - **Check Physical Preparation**
 - **Low:** Recovery and Adaptation Program
 - **Medium/High:** Check Stress Levels
 - **High:** Recovery and Adaptation Program
 - **Low/Medium:** Neurocognitive Enhancement Program (Modified Intensity)
 - **Check Sleep Quality**
 - **Poor:** Recovery and Adaptation Program
 - **Good:** Continue Assessment
- **Medium**
 - **Check Physical Preparation**
 - **Low/Medium:** Check Primary Goals
 - **Cognitive Focus:** Neurocognitive Enhancement Program
 - **Physical Focus:** Postural Restoration and Balance Program
 - **High:** Check Time Availability
 - **Limited:** Neurocognitive Enhancement Program
 - **Flexible:** Check Technical Foundation

- **Strong:** High-Performance Integration Program
 - **Weak:** Postural Restoration and Balance Program
 - **Check Recovery Capacity**
 - **Low:** Recovery and Adaptation Program
 - **Medium/High:** Continue Assessment
 - **High**
 - **Check Physical Preparation**
 - **Low:** Neurocognitive Enhancement Program
 - **Medium:** Check Primary Goals
 - **Neural Focus:** Neurocognitive Enhancement Program
 - **Balance Focus:** Postural Restoration and Balance Program
 - **High:** Check Technical Foundation
 - **Strong:** Check Recovery Capacity
 - **Adequate:** High-Performance Integration Program
 - **Limited:** Neurocognitive Enhancement Program
 - **Weak:** Postural Restoration and Balance Program
2. **Check Previous Injury History**
 - **Recent/Significant:** Recovery and Adaptation Program
 - **None/Resolved:** Continue with Selected Program

Decision Points Explanation

1. **Neural Readiness Assessment:** Evaluates cognitive status, nervous system recovery, and sleep quality (weighted at 30%).
2. **Physical Preparation:** Assesses movement competency, strength baseline, and endurance capacity (weighted at 25%).
3. **Lifestyle Compatibility:** Considers time availability, stress levels, and recovery resources (weighted at 25%).
4. **Goal Alignment:** Analyzes stated objectives, timeline requirements, and success metrics (weighted at 20%).

Program Outcomes

1. **Neurocognitive Enhancement Program:**
 - For users focused on cognitive improvement.
 - 3-4 sessions per week, 45-60 minutes each.
 - Targets: Knowledge workers, executives, students.
2. **Postural Restoration and Balance Program:**
 - For users needing posture correction and balance training.
 - 4-5 sessions per week, 30-45 minutes each.
 - Targets: Office workers, seniors, rehabilitation clients.
3. **High-Performance Integration Program:**
 - For advanced fitness practitioners.
 - 5-6 sessions per week, 60 minutes each.

title: Neuronline Fitness System Architecture Overview

tags: [neuronline, fitness, architecture, components, features, ui, data models, implementation, workflow]

Core Architecture

Project Structure

- **App Directory**
- **Components Directory**
- **Hooks Directory**
- **Lib Directory**
- **Public Directory**
- **Styles Directory**

Component Organization

- **Features Components**
- **Layout Components**
- **UI Components**
- **Common Components**

Key Features

Personalized Program Generation

- User Profiles and Metrics
- Fitness Goals and Preferences
- Neurological Testing Results
- Physical Capabilities and Limitations

Neural-Physical Integration

- Vestibular System Training
- Visual System Training
- Cognitive-Motor Integration
- Physical Conditioning

Progressive Development

- Foundation Development Phase
- Integration Phase
- Performance Enhancement Phase
- Mastery and Maintenance Phase

Adaptive Monitoring

- Data Collection Methods
- Adaptation Triggers
- Adjustment Mechanisms
- Feedback Channels

User Interface

Navigation Structure

- Primary Navigation (Bottom Tab Bar)
- Secondary Navigation (Top Header)
- Category Navigation (Filter Pills)

Layout Components

- Screen Container
- Header
- Feature Cards
- Category Grid
- Event Cards
- Bottom Navigation

Color Palette

- Primary Colors
- Secondary Colors
- Background Colors
- Text Colors
- Status Colors

Data Models

User Profiles

- Basic Info
- Health Metrics
- Body Measurements

Fitness Programs

- Program Metadata
- Schedule
- Weekly Structure
- Daily Programs

Exercises

- Exercise Metadata
- Type/Categorization
- Parameters
- Effects

Progress Tracking

- Completion Records
- Performance Metrics
- User Feedback

Implementation

Component Development

- Client Component Requirements
- Styling Approach
- Performance Optimizations

State Management

- Local State
- Server State
- Global State
- Persistent State

API Integration

- Data Fetching Strategy
- Error Handling
- Request Structure

Development Workflow

Version Control

- Branching Strategy
- Commit Message Format
- Pull Request Template

Quality Assurance

- Linting
- Formatting
- Testing
- Type Checking

Deployment Process

- Environments
 - Continuous Integration
 - Deployment Frequency
-

title: Building an Image Analyzer App with React and Node.js
tags: [react, node.js, tensorflow.js, image analysis]

Project Structure

Let's set up the project structure:

```
image-analyzer/  
  client/  
  server/
```

Frontend Setup (Client Folder)

1. Create a new React app:

```
npx create-react-app client  
cd client  
npm install axios @tensorflow/tfjs @material-ui/core
```

1. Create `src/App.js`:

```
import React, { useState } from 'react';  
import { Container, Paper, Button, Typography, CircularProgress } from '@material-ui/core';  
import * as tf from '@tensorflow/tfjs';  
import axios from 'axios';  
import './App.css';  
  
function App() {  
  const [selectedFile, setSelectedFile] = useState(null);  
  const [preview, setPreview] = useState(null);  
  const [analysis, setAnalysis] = useState(null);  
  const [loading, setLoading] = useState(false);  
  
  const handleFileSelect = (event) => {  
    const file = event.target.files[0];  
    setSelectedFile(file);  
  
    const reader = new FileReader();  
    reader.onloadend = () => {  
      setPreview(reader.result);  
    };  
  };  
}
```

```

    if (file) {
      reader.readAsDataURL(file);
    }
  };

  const analyzeImage = async () => {
    try {
      setLoading(true);

      // Load the MobileNet model
      const model = await tf.loadLayersModel('https://storage.googleapis.com/tfjs-models/tfjs/mobilenetv10.25224/model.json');

      // Create an HTML image element from the preview
      const img = new Image();
      img.src = preview;
      await img.decode(); // wait for image to load

      // Prepare the image for the model
      const tensor = tf.browser.fromPixels(img)
        .resizeNearestNeighbor([224, 224])
        .toFloat()
        .expandDims();

      // Get predictions
      const predictions = await model.predict(tensor).data();

      // Get top 5 predictions
      const top5 = Array.from(predictions)
        .map((p, i) => ({
          probability: p,
          className: IMAGENETCLASSES[i]
        })))
        .sort((a, b) => b.probability - a.probability)
        .slice(0, 5);

      setAnalysis(top5);
    } catch (error) {
      console.error('Error analyzing image:', error);
    } finally {
      setLoading(false);
    }
  };

  return (
    <Container maxWidth="md" className="App">
      <Paper elevation={3} style={{ padding: '2rem', marginTop: '2rem' }}>
        <Typography variant="h4" gutterBottom>
          Image Analyzer
        </Typography>

        <input
          accept="image/*"
          type="file"
          onChange={handleFileSelect}
          style={{ marginBottom: '1rem' }}
        />
      </Paper>
    </Container>
  );

```

```

{preview && (
  <div style={{ marginBottom: '1rem' }}>
    <img
      src={preview}
      alt="Preview"
      style={{ maxWidth: '100%', maxHeight: '300px' }}
    />
  </div>
)}

<Button
  variant="contained"
  color="primary"
  onClick={analyzeImage}
  disabled={!selectedFile || loading}
>
  Analyze Image
</Button>

{loading && (
  <div style={{ marginTop: '1rem' }}>
    <CircularProgress />
  </div>
)}

{analysis && (
  <div style={{ marginTop: '1rem' }}>
    <Typography variant="h6">Analysis Results:</Typography>
    <ul>
      {analysis.map((result, index) => (
        <li key={index}>
          {result.className}: {(result.probability * 100).toFixed(2)}%
        </li>
      ))}
    </ul>
  </div>
)}
</Paper>
</Container>
);
}

export default App;

```

title: Code Editing Assistant Capabilities
tags: [code, editing, tools, programming]

Role

You are a code editing assistant capable of fulfilling edit requests and engaging in discussions about code. Provide complete instructions or code lines using XML formatting.

Capabilities

- Create new files.
- Rewrite entire files.
- Perform partial search/replace modifications.
- Delete existing files.

Avoid placeholders like `...` or `// existing code here`. Provide complete lines or code.

Tools and Actions

1. **create:** Create a new file if it doesn't exist.
2. **rewrite:** Replace the entire content of an existing file.
3. **modify (search/replace):** For partial edits with `<search>` and `<content>`.
4. **delete:** Remove a file entirely (empty `<content>`).

Format for Repo Prompt's Diff Protocol

Describe your approach or reasoning here.

```
<file path/to/example.swift action=one_of_the_tools>
<change>
<description>Brief explanation of this specific change</description>
<search>
```

// Exactly matching lines to find

```
</search>
<content>
```

// Provide the new or updated code here. Do not use placeholders

```
</content>
```

Tools Demonstration

1. `<file path/NewFile.swift action=create>` Full file in ``<content>``.
2. `<file path/DeleteMe.swift action=delete>` Empty ``<content>``.
3. `<file path/ModifyMe.swift action=modify>` Partial edit with ``<search>`` and ``<content>``.
4. `<file path/RewriteMe.swift action=rewrite>` Entire file in ``<content>``.
5. `<file path/RewriteMe.swift action=rewrite>` Entire file in ``<content>``. No ``<search>`` required.

Format Guidelines

1. **Plan:** Begin with a `<Plan>` block explaining your approach.
2. **<file> Tag:** e.g., `<file path=Models/User.swift action=...>`. Must match an available tool.
3. **<change> Tag:** Provide `<description>` to clarify each change. Then `<content>` for new/modified code. Additional rules depend on your capabilities.
4. **modify:** `<search>` and `<content>`: Provide code blocks enclosed by `.`. Respect indentation exactly, ensuring the `<search>` block matches the original source down to braces, spacing, and any comments. The new `<content>` will replace the `<search>` block and should fit perfectly in the space left by its removal.
5. **modify:** For changes to the same file, ensure that you use multiple change blocks, rather than separate file blocks.
6. **rewrite:** For large overhauls, omit `<search>` and put the entire file in `<content>`.
7. **create:** For new files, put the full file in `<content>`.
8. **delete:** Provide an empty `<content>`. The file is removed.

Code Examples

Example: Search and Replace (Add email property)

Add an email property to `User` via search/replace.

```
struct User {  
  let id: UUID  
  var name: String  
}
```

```
</search>  
<content>
```

```
struct User {  
  let id: UUID  
  var name: String  
  var email: String  
}
```

```
</content>
```

change> file>

Example: Negative Example - Mismatched Search Block

Demonstrate how a mismatched search block leads to failed merges.

```
foo() {  
  Bar()  
}  
  
</search>  
<content>  
  
// Corrected code goes here  
  
</content>
```

change> file>

title: Building the Neurofitness Enterprise Application with Next.js 13
tags: [next.js, typescript, tailwind css, supabase, neurofitness]

Overview

The Neurofitness Enterprise Application is a Next.js 13-based fitness platform that leverages TypeScript, Tailwind CSS, and Supabase for a comprehensive neurofitness experience. This application is designed with a robust architecture to support various features and functionalities.

System Architecture

Frontend

- **Framework:** Utilizing Next.js 13 with the App Router for efficient routing.
- **Styling:** Tailwind CSS for responsive design.
- **State Management:**
 - Local: React Hooks
 - Global: Context API
 - Server: Server Actions

Components

- **Organization:** Feature-based structure
- **Design:** Atomic design principles
- **Styling:** Tailwind modules for component-specific styles

Backend

- **Database:** Powered by Supabase PostgreSQL, featuring schemas for:

- User data
- Content management
- Training metrics
- Rewards
- **Key Features:**
 - Full-text search
 - Real-time subscriptions
 - Row-level security
 - Automated backups
- **Authentication:** Managed through Supabase Auth
- **Storage:** Supabase Storage for file management
- **Real-time Capabilities:** Supabase Realtime for live data updates

Database Schema

Schemas

- **User Data:**
 - Tables: `users`, `biometrics`, `fitnesspreferences`, `availabledays`, `healthrestrictions`, `usergoals`
 - Views: `activeusersview`
- **Content:**
 - Tables: `categories`, `subcategories`, `neuroeffects`, `equipment`, `courses`, `videos`
 - Views: `videopopularityview`
- **Training:**
 - Tables: `usercourses`, `fitnessplans`, `weeklyplans`, `dailyworkouts`, `workoutactivities`
 - Views: `userworkoutscheduleview`

Application Structure

Root Structure

- **App Groups:**
 - **Auth:** Routes for `/login`, `/register`, `/callback`, `/signout` with `AuthLayout`
 - **Dashboard:** Routes for `/dashboard`, `/analytics`, `/progress` with `DashboardLayout`
 - **Workouts:** Routes for `/workouts`, `/workout-player`, `/history` with `WorkoutLayout`

Components

- **Features:**
 - Achievements: `AchievementGrid`, `RewardsDisplay`
 - Workouts: `WorkoutCard`, `WorkoutProgress`
 - Neuro: `NeuralTest`, `CourseCard`
- **Layout:**
 - Navigation: `MainNav`, `DashboardNav`

- Containers: `DashboardShell`, `ContentWrapper`

Data Flow

API

- **Routes:**
 - Base: `/api`
 - Handlers:
 - Auth: `login`, `register`, `callback`
 - Workouts: `list`, `detail`, `progress`
 - Neuro: `test`, `courses`, `results`

State Management

- **Contexts:**
 - Auth: `AuthContext`
 - Workout: `WorkoutContext`
 - Rewards: `RewardsContext`

Business Logic

Neurofitness Programs

- **Neurocognitive:**
 - Focus: Neural pathway activation
 - Components: `Vestibular`, `Visual`, `Coordination`
 - Progression: 4-week cycles
- **Postural:**
 - Focus: Alignment and stability
 - Components: `Mobility`, `Strength`, `Balance`
 - Progression: 3-week cycles

Adaptation Metrics

- **Neural:**
 - Metrics: Reaction time, Movement accuracy, Cognitive load

This structured approach ensures that the Neurofitness Enterprise Application is scalable, maintainable, and equipped to deliver a rich user experience.

```

---
title: Exploring the Neuron Fit Next App Structure
tags: [neuron, fit, nextjs, app-structure, components]
---

## Project Overview

The Neuron Fit Next App is structured to facilitate a modular and scalable architecture.
Below is a detailed breakdown of its directory structure, highlighting key components and
their functionalities.

## Directory Structure

### Core Components

- **app/**
  - **achievements/**: Contains `page.tsx` for displaying user achievements.
  - **neurobot/**: Manages the neurobot interface with `page.tsx`.
  - **neurofitness-categories/**: Displays fitness categories via `page.tsx`.
  - **notifications/**: Handles user notifications with `page.tsx`.
  - **privacy/**: Manages privacy settings in `page.tsx`.
  - **profile/**: User profile management through `page.tsx`.
  - **shop/**: E-commerce functionalities in `page.tsx`.
  - **statistics/**: Displays user statistics via `page.tsx`.
  - **subscription/**: Manages subscriptions with `page.tsx`.
  - **workout-history/**: Tracks workout history in `page.tsx`.
  - **workout-player/**: Manages workout playback through `page.tsx`.
  - **workouts/**: Displays available workouts via `page.tsx`.
  - **globals.css**: Global styles for the application.
  - **layout.tsx**: Main layout component.
  - **page.tsx**: Entry point for the app.

### UI Components

- **components/**
  - **dashboard/**:
    - `DashboardStats.tsx`: Displays dashboard statistics.
    - `ProgressOverview.tsx`: Provides an overview of user progress.
    - `WorkoutSummary.tsx`: Summarizes workout data.
  - **exercise/**:
    - `ExerciseGrid.tsx`: Displays exercises in a grid format.
  - **features/neuro/**:
    - `ExercisePlayer.tsx`: Manages exercise playback.
    - `index.ts`: Entry point for neuro features.
    - `NeurofitnessCategoriesView.tsx`: Displays neuro fitness categories.
  - **neurotest/**:
    - `Onboarding.tsx`: Handles onboarding for neuro tests.
    - `Results.tsx`: Displays test results.
    - `TestInterface.tsx`: Manages the test interface.
  - **progress/**:
    - `Achievements.tsx`: Displays achievements.
    - `ProgressCharts.tsx`: Visualizes progress data.
    - `ProgressMetrics.tsx`: Provides metrics on user progress.
  - **schedule/**:
    - `Calendar.tsx`: Displays a calendar for scheduled workouts.
    - `ScheduledWorkouts.tsx`: Manages scheduled workouts.
  - **ui/**: Contains various UI components such as buttons, dialogs, and forms.

```

Hooks and Utilities

- ****hooks/****:
 - ``use-mobile.tsx``: Custom hook for mobile responsiveness.
 - ``use-toast.ts``: Manages toast notifications.
 - ``useAchievements.ts``: Custom hook for achievements management.
 - ``useRouter.ts``: Custom router hook for navigation.
- ****lib/****:
 - ``auth.ts``: Authentication utilities.
 - ``points.ts``: Points management utilities.
 - ``utils.ts``: General utility functions.

Application Entry

- ****my-app/****: Contains the main application components and structure, mirroring the ``components`` directory for modularity.

Conclusion

The Neuron Fit Next App is designed with a clear separation of concerns, allowing for easy maintenance and scalability. Each component is purpose-built, ensuring that the application remains efficient and user-friendly.

You are an expert AI coding assistant. Your task is to help users with any coding-related request.

IMPORTANT: Before suggesting any changes:

1. ALWAYS read the current content of the file first
2. Analyze the content to understand the current state
3. Only specify the exact changes needed
4. Do not include unchanged content

When you see files marked with [ATTACHMENT], these are additional files provided by the user for context. Use them to:

1. Understand code patterns and styles to maintain consistency
2. Reference implementations or examples
3. Extract relevant code snippets or configurations
4. Follow similar patterns when generating new code

Operation types can include but are not limited to:

- `create_file`: Create a new file (requires complete content)
- `edit_file`: Specify only the changes needed
- `remove_file`: Delete a file
- `rename_file`: Rename/move a file (requires `new_path`)
- `remove_directory`: Delete a directory (requires recursive boolean)

Guidelines for operations:

1. First read and analyze the existing file

2. When a file needs multiple operations (e.g., content changes AND rename):
 - First use `edit_file` to modify the content
 - Then use `rename_file` to move/rename the file
 - Each operation should be a separate entry in the operations array
3. For modifying content:
 - Only specify the exact text to change
 - Use the "changes" array to list each modification
 - For simple text replacements (e.g., replacing all instances of "nginx" with "apache"), include just one change with the exact text to replace
 - For complex code changes, include each specific change separately
4. For renaming files:
 - Use the `rename_file` operation type
 - Specify both path (current) and new_path (target)
 - If content also needs to change, do that in a separate `edit_file` operation first

IMPORTANT:

- ALWAYS read the file content first
 - ONLY specify the changes needed
 - NEVER return unchanged content
 - ALWAYS preserve the file's existing style and formatting
 - When both renaming and editing a file, do BOTH operations in the correct order
 - When attachments are provided, use them as reference for code style and patterns.
-

description: Cursor Rules Location
globs: *.mdc

Cursor Rules Location

Rules for placing and organizing Cursor rule files in the repository.

actions:

- type: reject
conditions:
 - pattern: "`^(?!\\.\\.\\.cursor\\rules\\.*\\.mdc$)"`"
message: "Cursor rule files (.mdc) must be placed in the `.cursor/rules` directory"
- type: suggest
message: |
When creating Cursor rules:
 1. Always place rule files in `PROJECT_ROOT/.cursor/rules/`:

```
.cursor/rules/  
├── your-rule-name.mdc  
├── another-rule.mdc  
└── ...
```

2. Follow the naming convention:

- Use kebab-case for filenames
- Always use .mdc extension
- Make names descriptive of the rule's purpose

3. Directory structure:

```
PROJECT_ROOT/  
├── .cursor/  
│   └── rules/  
│       ├── your-rule-name.mdc  
│       └── ...  
└── ...
```

4. Never place rule files:

- In the project root
- In subdirectories outside .cursor/rules
- In any other location

examples:

- input: |

Bad: Rule file in wrong location

```
rules/my-rule.mdc  
my-rule.mdc  
.rules/my-rule.mdc
```

Good: Rule file in correct location

```
.cursor/rules/my-rule.mdc  
output: "Correctly placed Cursor rule file"
```

metadata:

priority: high

version: 1.0

</rule>

```

{  "name": "Neurofitness Enterprise Application",
  "version": "3.0.0",
  "description": "Next.js 13+ enterprise fitness application with TypeScript, Tailwind CSS, Supabase integration, and
  comprehensive neurofitness features.",
  ".systemArchitecture": {
    "frontend": {
      "framerow": { name: "App Router", routing: true }, styling: {"tailwindcss": true}, stateManagement: { local: "React
      Hooks", global: "Context API", server: "Server Actions" } },
      "backend": { database: { provider: "Supabase PostgreSQL", schemas: ["user_data", "content", "training", "metrics",
      "rewards"], features: ["Full-text search", "Real-time subscriptions", "Row-level security", "Automated backups"] },
      authentication: "Supabase Auth", storage: "Supabase Storage", realtime: "Supabase Realtime" }
    },
    ".databaseSchema": {
      "user_data": {
        ".tables": [ "users", "biometrics", "fitness_preferences", "available_days", "health_restrictions", "user_goals" ],
        ".views": ["active_users_view"]
      },
      "content": {
        ".tables": [
          categories",
          subcategories",
          neuro_effects",
          equipment",
          courses",
          videos"
        ],
        ".views": ["video_popularity_view"]
      },
      "training": {
        ".tables": [
          user_courses",
          fitness_plans",
          WEEKLY PLANS",
          daily_workouts",
          workout_activities"
        ],
        ".views": ["user_workout_schedule_view"]
      }
    },
    ".appStructure": {
      "root": {
        "app": {
          groups: "(auth)" => routes /login etc., (dashboard)=>routes /analytics etc., (workouts) => routes /workouts etc.
        }",
        "components": {
          ".features: Achievements(rewards display), workouts(workout cards progress),
          .neuro: Neural test course card)
          /layout: Navigation(main nav dashboard containers).
        },
        ".dataFlow": {
          api: base/ handlers auth list detail progress;
          state: contexts auth workout rewards;
          _businessLogic: neuro fitness programs integration metrics development maintenance;
        }
      }
    }
  }
}

```

```

{
  "name": "Decision Tree for Neurofitness Program Recommendations",
  "type": "json",
  "children": [
    {
      "@context": "https://www.crisp.com/context/base.org Kratos/MetaModel/2024-11-28 - Decision Trees",
      "@id": "#_decision_tree_for_neurofitness_program_recommendations",
      "name": "System Architecture Overview",
      "category": "application/json",
      "schemaVersion": "http://example-com/example-schema.json v1.0",
      "description": "A decision tree that helps determine the most appropriate program recommendations based on user goals."
    },
    {
      "@context": null,
      "@id": "",
      "name": "Program Selection Flowchart",
      "category": "flowChart",
      "imageMap": [],
      "requirements": [
        {
          "@context": "iso/stds",
          "@id": "ISO/IEC TS/9356-7",
          "@title": "Neuroonline Fitness System Performance Requirements",
          "@ALaperfNumber": "Performance Metric #3: Cognitive Enhancement and Integration Metrics",
          "@ALaperfScope": "Fitness Functionality",
          "@ALaperfSub scope": "User Experience",
          "@ALaperfRequirements": [
            {"@Title": "Integration Phase Progression"}, {"@Title": "Adherence to Timelines"}, {"@Title": "Secondary Goals Monitoring"}
          ]
        }
      ]
    }
  ]
}

```

I'll create a comprehensive system architecture document covering all aspects of your application.

```

{
  "name": "Neurofitness Enterprise Application",
  "version": "3.0.0",
  "description": "Next.js 13+ enterprise fitness platform with advanced features.",
  "_etag": "...", // Include unique ID from git log if available

  "systemArchitecture": {
    "frontend": {
      "framework": "nextjs",
      "routing": true, **["App Router"]**
      "styling": ["tailwindcss"],
      "stateManagement": {
        "local": "react hooks",
        "global": "context api",
        "server": "server actions"
      }
    }
  },

```

```

"backend": {
  "database": {
    "provider": "supabase postgresql",
    "schemas": [
      "user_data",
      "content",
      "training",
      "metrics",
      "rewards"
    ],
    "features": [
      "full-text search",
      "real-time subscriptions",
      "row-level security",
      "automated backups"
    ]
  },
  "authentication": "supabase auth",
  "storage": "supabase storage",
  "realtime": "supabase realtime"
},

"dataFlow": {
  "api": "/api handlers for authentication, workouts, neuro programs etc.",

  "state": {
    "contexts": {
      "auth": "auth context",
      "workout": "workout contexts",
      "rewards": " rewards context"
    },

    "persistentState": {
      "user profile": ...,
      "last workout plan update": ...
    }
  },

  "businessLogic": {
    "neuro fitness Programs integration logic",
    "adaptation metrics tracking",
    "development standards compliance",
    "maintenance procedures"
  }
}

"architectureStructure": {
  "componentOrganization": {
    "components": [
      "(features)": [achievements (reward display), workouts (card
progress)],

      "(layout)": main nav dashboard containers

      "(/navigation)": groups: auth, dashboard, workouts
    ]
  },

```

```

    "keyFeatures": [
      full-body warming exercise program,
      progressive overload in exercises and sets,
      recovery phase management

      stress reduction through mindfulness training,
      balanced mobility techniques,
      posture correction options
    ]

    "implementationStandards": {
      "codeStyle": { "typescript": true },
      "testing": { "unit": true, "test-coverage": .95},
      "security": { "dbms": true}
    }
  }

  "deploymentPlan": {
    "integrations": {
      "database": "daily data backup with 7-day retention period",
      "optimization": {
        "sorting/organizing": daily Workout Plan Review tool,
        "compression": content compression strategy
      }
    }
  }
}

```